Scanner-Noise Cleanup Pipeline for 3-D Scans

(pre-mesh documentation)

Gaurang Chauhan

About this documentation. All procedures, explanations, and figures herein reflect *my* study of scanner-noise removal. I consulted several research articles and online resources, then wrote the concepts with my understanding, combined them with the necessary mathematics, and produced concise step-by-step notes. Every diagram was hand-drawn on a tablet. The result is a self-contained reference on cleaning point-cloud data before meshing and further analysis.

Scanner-Noise Cleanup Pipeline for 3-D Scans

This note (my understanding so far) explains, step by step, how we can

- delete distant outliers,
- smooth rough bumps(spikes) with MLS,
- polish and tidy the mesh,

1 Why cleanup?

- **These noises arise from scanner limitations**
  - **Sensor precision** — hardware accuracy, distance baseline, alignment, calibration.
  - **Surface (skin) reflectivity** — shiny or dark patches that return weak or distorted signals.
  - **Occlusions** — body regions hidden from the scanner's line of sight (e.g. armpits), leaving holes.
- **Raw-scan artefacts**
  - rough-surface noise (tiny bumps),
  - distant specks (outliers),
  - degenerate faces (zero-area triangles),
  - unreferenced vertices (vertices referenced only by degenerate faces).

(Rachakonda et al., 2019; Weyrich et al., 2004; Daneshmand et al., 2018)

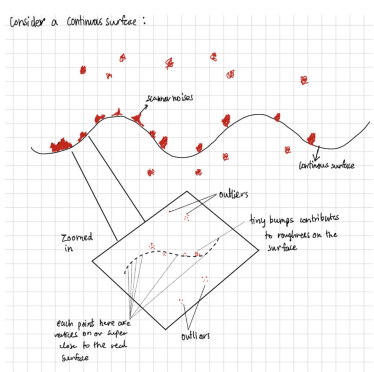


Figure 1: Continuous surface (black), dense on-surface points, rough-surface spikes (nearby red noise to be MLS-smoothed), and distant outliers (isolated red specks removed by the radius-count test).

2 Outlier Removal (radius–count test)

- Fix a radius ε for the neighbourhood ball (e.g. $\varepsilon = 10^{-4}$, 10^{-6}, or 10^{-8}).
- For each vertex p in the point cloud, count how many other vertices fall inside the closed ball

$$B_\varepsilon(p) = \{\, v \mid \|p - v\| \le \varepsilon \}.$$

  Require this count to be at least m.

Rule.

$$\text{keep vertex } p \quad \Longleftrightarrow \quad \Big|\big\{ v \;\big|\; \|p - v\| \le \varepsilon \big\}\Big| \;\ge\; m,$$

where $|\cdot|$ denotes cardinality.

Intuition / Rationale. True outliers are isolated points (or tiny clusters), so they do not have m neighbours within the chosen radius and are discarded by this neighbour-count test.
(Rusu and Cousins, 2011; Point Cloud Library, "RadiusOutlierRemoval"; Open3D, "Point Cloud Outlier Removal")

Note : Log every removal step, record the IDs and diagnostics for each outlier vertex. So the cleaning process is fully traceable and reproducible.

What comes next. With the distant outliers discarded, the remaining cloud still contains rough-surface spikes that must be smoothed. To tackle those spikes we first need a fast way to retrieve local neighbourhoods. The next pages therefore introduce

a) **KD-tree terminology** — key terms used throughout;

b) an **illustration of KD-tree structure**;

c) the detailed methodology:

- building the KD-tree, and
- performing k-nearest-neighbour (k-NN) queries.

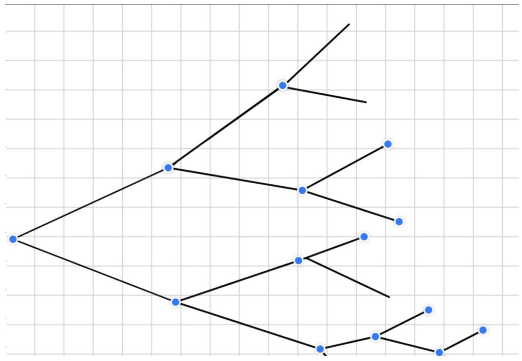
Those k-NN queries will feed directly into the MLS projection step that smooths the rough bumps before we proceed to meshing and segmentation.

KD-Tree Terminology

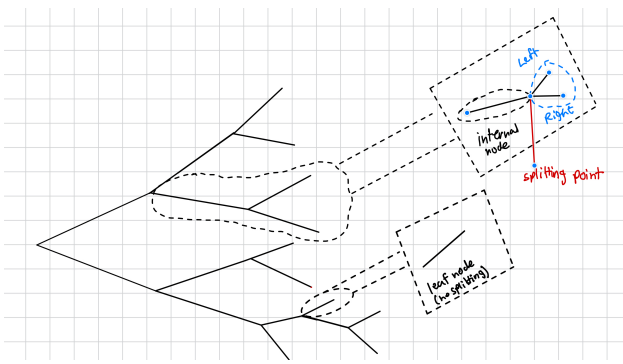
Root node	The first node that initially holds all points before any split.
Internal node	A node that has been split into two children and does not store raw points itself.
Leaf node	A node containing at most ℓ points (where ℓ is `leaf_size`); splitting stops here and all its points are stored.
Splitting axis	The coordinate axis $(x, y, z, \dots)$ chosen for the current split, cycling through axes level by level.
Median split point	The point whose coordinate is the $\lfloor n/2 \rfloor$-th smallest along the splitting axis, balancing the two subtrees.
Bounding box	$[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}]$ an axis-aligned rectangular prism that encloses all points in a node.Inside the box live all valid (x, y, z) combinations. **Note:** Every internal node or leaf has its own bounding box.
Wall distance	The shortest Euclidean distance from the query point Q to any face of a node's bounding box (zero if Q lies inside).
Sibling branch	The child subtree that was *not* followed during the descent from the root to the leaf containing Q.
Point-heap (max-heap)	A priority queue of the current k nearest candidate points; the farthest of these appears at the top.
Box-heap (min-heap)	A priority queue of unexplored nodes ordered by their wall distance; the nearest box is at the top.

(Bentley, 1975; Friedman et al., 1977; SciPy Developers, "KDTree")

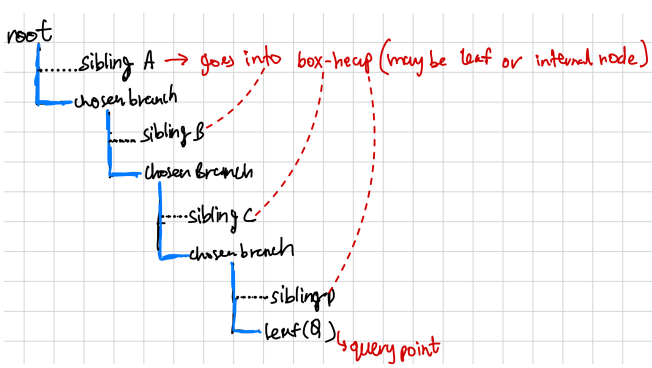
KD-Tree Illustration



(a) Overall KD-tree structure. Blue dots mark the nodes; black line segments illustrate the splitting hyperplanes (projected as edges in this sketch).



(b) Overview of a KD-tree with two call-outs (dashed rectangles) that zoom into specific regions. The upper call-out enlarges an **internal node**: the red segment marks its splitting point, and the blue dashed loop highlights how the split yields *left* and *right* sub-branches. The lower call-out enlarges a **leaf node**, which stores raw points and is not split further.



(c) Branch-and-bound descent in a KD-tree. The solid path traces the search from the *root* to the *query-leaf*. At each level, the sibling subtree (dashed arrow) is queued in the *box-heap* for later exploration.

Building the KD-Tree (Construction Phase)

1. **Initial Root and Axis Selection:** Begin with all points at the root node. The splitting axis cycles through dimensions ($x \to y \to z$, then repeat) as you descend each level.

For instance, begin on the x-axis: sort the points in ascending x-coordinate order and choose the median as the pivot. Points with x-coordinate $\leq$ that pivot go to the left child, the rest to the right. At the next level (unless the node is already a leaf) switch to the y-axis ; then (unless the node is already a leaf) z-axis , and so on.

2. **Median Split on Chosen Axis:** Sort by the current axis (or use a median-finding algorithm) and pick the median point as the pivot. This splits the node's points into two roughly equal halves.

Points with coordinate $\leq$ median OR $\lfloor n/2 \rfloor$-th value go to the left child.

Points with coordinate $>$ $\lfloor n/2 \rfloor$-th value go to the right child.

(This yields a balanced KD-tree in ideal cases.)

3. **Store Node's Bounding Box:** For each node, compute its axis-aligned bounding box: $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}]$. This bounding box (the smallest rectangular prism containing the points) will be used later for efficient pruning. The "walls" of this box define the minimum distance a query point would have to travel to reach any point in the node – called the wall distance to the node.

4. **Recurse or Terminate:** Recursively repeat steps 1–3 for the left and right subsets of points (moving to the next axis in cycle for each level). Stop splitting a node when it contains $\leq$`leaf_size` points(for example, 30). Such a node is marked as a leaf and holds those points without further division. The `leaf_size` threshold is a practical implementation detail to limit tree depth and switch to brute- force search within leaves .

(Bentley, 1975; SciPy Developers, "KDTree")

k-Nearest Neighbor Query Using the KD-Tree

1. **Initial Descent to a Leaf**

   - **Traverse down:** Start at the root and follow the splitting rules down to a leaf node, as if you were inserting the query point Q into the tree . At each internal node, compare Q's coordinate on that node's axis to the node's split value, and go left or right accordingly. This descent ends at a leaf node (the region in which Q would lie).
   - **Evaluate leaf points:** Once at the leaf, compute the distance from Q to each point in that leaf. Insert all these distances (together with references to the actual points) into a *max-heap* called the *point-heap*. This heap stores the current closest points; it is organised as a max-heap so that the *worst* (farthest) neighbour sits conveniently at the top. If the leaf contains ℓ points and $\ell \geq k$, you may keep only the closest k of them in the heap and discard the rest.

     Note: The symbols differ in meaning -ℓ denotes `leaf_size`, the maximum points stored in any leaf, whereas k is the number of nearest neighbours we wish to return for the query point. They need not be equal.
   - **Track sibling subtrees:** During the descent, note each decision where you went left or right. For every such internal node there is an *untaken sibling branch*. Each of these sibling subtrees could contain points that are closer to Q than those already found. Push every untaken subtree into a *min-heap* called the *box-heap*, using as the key the minimum possible distance from Q to that subtree's bounding box. (If Q lies *outside* the bounding box, the key is the distance from Q to the nearest face of the box; if Q is *inside* the box, this distance is 0.) The box-heap therefore orders subtrees by how promising they are: a lower *wall distance* means a more promising subtree.

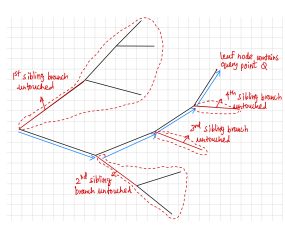


Figure 3: An example to visualize; observe Blue path, the search route from the root to the leaf that contains the query point Q. Each dashed red outline marks an *untouched sibling branch*. As soon as the algorithm skips that branch , the entire sibling subtree,together with all of its descendants, is pushed into the box-heap. The labels "1st sibling", "2nd sibling", etc. show the order in which these branches are enqueued.

(Friedman et al., 1977; SciPy Developers, "KDTree")

2. **Fill Phase :** (A leaf node stores at most $\ell =$ `leaf_size` points and at least one point. When the query point Q lands in a leaf whose occupancy ℓ_Q satisfies $\ell_Q < k$, the initial scan can seed the *point-heap* with only ℓ_Q candidates—fewer than the k neighbours we ultimately need to report. The *fill phase* is therefore invoked *only* in this situation: we expand additional subtrees, adding their points to the point-heap, until the heap contains at least k points for this specific query. (If the leaf already holds k or more points, the heap is immediately full and the algorithm skips straight to the pruning phase.)

- **Expand closest subtree:** Examine the *box-heap* and pop the subtree with the smallest bounding-box distance (i.e. the most promising unexplored region). Open this node unconditionally because we still need more points.
- *If the popped item is an internal node*, retrieve its two children. Compute each child's bounding-box distance to Q and push both children onto the box-heap. (We are effectively descending one level deeper in that subtree.)
- *If the popped item is a leaf node*, compute the distance from Q to every point in that leaf and insert each distance into the *point-heap*. If the point-heap grows beyond size k, remove the farthest point (the max-heap root) so that only the best k candidates remain.
- **Repeat until k achieved:** Continue expanding subtrees, always choosing the next closest subtree from the box-heap, and adding points until the point-heap contains k points. During this phase we perform *no pruning* based on distance; we are simply accumulating k points as quickly as possible. Until the point-heap holds k points, we treat its "worst distance" as effectively infinite, so every subtree is worth exploring. In other words, the first k points encountered are added outright.

Result of fill phase: Once the point-heap is filled with k points, we have an initial set of k nearest-neighbour candidates. The largest distance in this heap (call it $d_{\max}$) is the current radius, any further neighbour must lie at most $d_{\max}$ away from Q to be competitive. (Friedman et al., 1977; SciPy Developers, "KDTree")

3. **Prune Phase (Refine and Prune Search)**

Now that we have k neighbours, we enter the pruning phase to be sure they are the *true* k nearest and that no closer points exist in unexplored regions:

- **Check next region:** Look at the top of the box-heap (the unexplored subtree with the smallest bounding-box distance from Q). Let d_{box} be this minimum distance. If $d_{\text{box}} \geq d_{\max}$ (the farthest distance in our neighbour heap), this subtree *cannot* contain any point closer than our worst neighbour found so far. Therefore, we can **prune** this subtree,remove it from consideration without exploring its points.

  Note: This pruning step is precisely what gives the KD-tree search its speed advantage over a brute-force Euclidean scan, because entire subtrees are discarded in one comparison instead of evaluating every point they contain.

- **Explore if promising:** If $d_{\text{box}} < d_{\max}$, the region *might* contain a closer neighbour. Pop that subtree from the box-heap and **explore** it:
  - If it is an *internal node*, push its children (each with their box-distance keys) into the box-heap for later consideration.
  - If it is a *leaf node*, compute distances from Q to all its points. For every point closer than the current worst neighbour, insert its distance into the point-heap. If the heap size exceeds k, remove the new farthest point to maintain size k. After any insertion, update $d_{\max}$ to the new worst distance in the point-heap (which may decrease).

- **Continue loop:** Repeat the above step. Always compare the smallest remaining box distance against the current $d_{\max}$. Prune subtrees that are too far, and fully explore subtrees that are potentially nearer than the worst current neighbour. This loop ends when *all* remaining unexplored subtrees are pruned (each has a min-distance $> d_{\max}$) or the box-heap is empty. At that point the point-heap definitively contains the k nearest neighbours of Q, and $d_{\max}$ is the true distance to the k-th nearest neighbour.

- **Result:** The algorithm terminates with the point-heap holding exactly k points,the nearest neighbours to the query point Q (and their distances). This outcome is exact: no closer points have been missed, thanks to the pruning criterion above. (Friedman et al., 1977; Bentley, 1975)

3 Moving Least-Squares (MLS) Projection for Noise Reduction

After the KD-tree delivers the k nearest neighbours for each scan point, we apply MLS "smooth bumping" to suppress scanner noise.
(Alexa et al., 2003; Levin, 2004; Fleishman et al., 2005; Point Cloud Library, "Smoothing and Normal Estimation")

Intuition : Noisy points often protrude above the real surface. By fitting a smooth local surface to their k nearest neighbours and projecting each point onto that surface, we drag every rough-surface spike back toward the underlying geometry.

directly into meshing or further analysis.

MLS projection step. For each sample point $\mathbf{p}$ in the raw cloud we

a) query its k nearest neighbours $\{\mathbf{p}_1, \ldots, \mathbf{p}_k\}$ with the KD-tree;

b) fit a local polynomial surface $f(\mathbf{x}) = 0$ by moving least-squares so that $f(\mathbf{p}_i) \approx 0$ for all neighbours; and

c) *project* $\mathbf{p}$ *itself* onto that surface via the orthogonal projection

$$\mathbf{p}' \;=\; \mathbf{p} \;-\; f(\mathbf{p})\,\frac{\nabla f(\mathbf{p})}{\|\nabla f(\mathbf{p})\|^2}.$$

Only the point being smoothed is moved; its k neighbours remain unchanged, they merely define the best-fitting local surface. Because protruding ("rough-surface") samples have a larger signed distance $|f(\mathbf{p})|$, the projection removes that extra height and places the sample closer to the real surface, reducing noise before meshing or analysis.

Safeguard for small gradients. If the fitted patch is almost flat at $\mathbf{p}$ we have $\|\nabla f(\mathbf{p})\| < \gamma$ for some numerical threshold γ (e.g. 10^{-6}). In that case we *skip* the projection and leave the point unchanged:

$$\mathbf{p}' = \begin{cases} \mathbf{p} & \text{if } \|\nabla f(\mathbf{p})\| < \gamma, \\ \mathbf{p} - f(\mathbf{p})\,\dfrac{\nabla f(\mathbf{p})}{\|\nabla f(\mathbf{p})\|^2} & \text{otherwise.} \end{cases}$$

This avoids division by a near-zero number *and* leaves truly on-surface samples untouched, while still projecting rough-surface spikes that have significant signed distance or gradient magnitude.

4 Degenerate–Face Removal

For every triangular face $f = (v_1, v_2, v_3)$ compute its signed area

$$A(f) = \tfrac{1}{2} \left\| (v_2 - v_1) \times (v_3 - v_1) \right\|.$$

Rule.

$$\text{discard face } f \iff A(f) < \varepsilon$$

Faces whose three vertices are collinear ($A(f) = 0$) or almost collinear ($A(f) < \varepsilon$) are thus removed.

Orphan-vertex sweep. After deleting all degenerate faces, remove every vertex that is no longer referenced by any face; these are *orphan vertices* created solely by the discarded faces.

- This two-step cleanup guarantees that each remaining face has positive area and every vertex participates in at least one non-degenerate triangle.
- The resulting mesh is free of zero-area artefacts

(Botsch et al., 2010; CGAL Project, "Polygon Mesh Processing")

Note : Log every removal step, record the IDs and diagnostics for each degenerate face, and orphan vertex so the cleaning process is fully traceable and reproducible.

Works Cited

Alexa, Marc, Johannes Behr, Daniel Cohen-Or, Shachar Fleishman, David Levin, and Claudio T. Silva. "Computing and Rendering Point Set Surfaces." *IEEE Transactions on Visualization and Computer Graphics*, vol. 9, no. 1, 2003, pp. 3-15. https://doi.org/10.1109/TVCG.2003.1175093.

Bentley, Jon Louis. "Multidimensional Binary Search Trees Used for Associative Searching." *Communications of the ACM*, vol. 18, no. 9, 1975, pp. 509-517. https://doi.org/10.1145/361002.361007.

Botsch, Mario, Leif Kobbelt, Mark Pauly, Pierre Alliez, and Bruno Levy. *Polygon Mesh Processing*. A K Peters/CRC Press, 2010. https://doi.org/10.1201/b10688.

CGAL Project. "Polygon Mesh Processing: Geometric Repair and Combinatorial Repairing." *CGAL Documentation*, https://doc.cgal.org/latest/Polygon_mesh_processing/. Accessed 15 May 2026.

Daneshmand, Morteza, Ahmed Helmi, Egils Avots, Fatemeh Noroozi, Fatih Alisinanoglu, Hasan Sait Arslan, Jelena Gorbova, Rain Eric Haamer, Cagri Ozcinar, and Gholamreza Anbarjafari. "3D Scanning: A Comprehensive Survey." *arXiv*, 2018. https://doi.org/10.48550/arXiv.1801.08863.

Fleishman, Shachar, Daniel Cohen-Or, and Claudio T. Silva. "Robust Moving Least-Squares Fitting with Sharp Features." *ACM Transactions on Graphics*, vol. 24, no. 3, 2005, pp. 544-552. https://doi.org/10.1145/1073204.1073227.

Friedman, Jerome H., Jon Louis Bentley, and Raphael Ari Finkel. "An Algorithm for Finding Best Matches in Logarithmic Expected Time." *ACM Transactions on Mathematical Software*, vol. 3, no. 3, 1977, pp. 209-226. https://doi.org/10.1145/355744.355745.

Levin, David. "Mesh-Independent Surface Interpolation." *Geometric Modeling for Scientific Visualization*, edited by Guido Brunnett, Bernd Hamann, Heinrich Muller, and Lars Linsen, Springer, 2004, pp. 37-49. https://doi.org/10.1007/978-3-662-07443-5_3.

Open3D. "Point Cloud Outlier Removal." *Open3D Documentation*, https://www.open3d.org/docs/latest/tutorial/Advanced/pointcloud_outlier_removal.html. Accessed 15 May 2026.

Point Cloud Library. "RadiusOutlierRemoval." *PCL Documentation*, https://pointclouds.org/documentation/classpcl_1_1_radius_outlier_removal.html. Accessed 15 May 2026.

Point Cloud Library. "Smoothing and Normal Estimation Based on Polynomial Reconstruction." *PCL Tutorials*, https://pcl.readthedocs.io/projects/tutorials/en/master/resampling.html. Accessed 15 May 2026.

Rachakonda, Prem K., Bala Muralikrishnan, and Daniel S. Sawyer. "Sources of Errors in Structured Light 3D Scanners." *National Institute of Standards and Technology*, 15 Apr. 2019. https://www.nist.gov/publications/sources-errors-structured-light-3d-scanners.

Rusu, Radu Bogdan, and Steve Cousins. "3D Is Here: Point Cloud Library (PCL)." *2011 IEEE International Conference on Robotics and Automation*, IEEE, 2011, pp. 1-4. https://doi.org/10.1109/ICRA.2011.5980567.

SciPy Developers. "KDTree." *SciPy Documentation*, https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.KDTree.html. Accessed 15 May 2026.

Weyrich, Tim, Mark Pauly, Richard Keiser, Simon Heinzle, Sascha Scandella, and Markus Gross. "Post-Processing of Scanned 3D Surface Data." *Eurographics Symposium on Point-Based Graphics*, 2004, pp. 85-94. https://dl.acm.org/doi/10.5555/2386332.2386347.

-- The End --